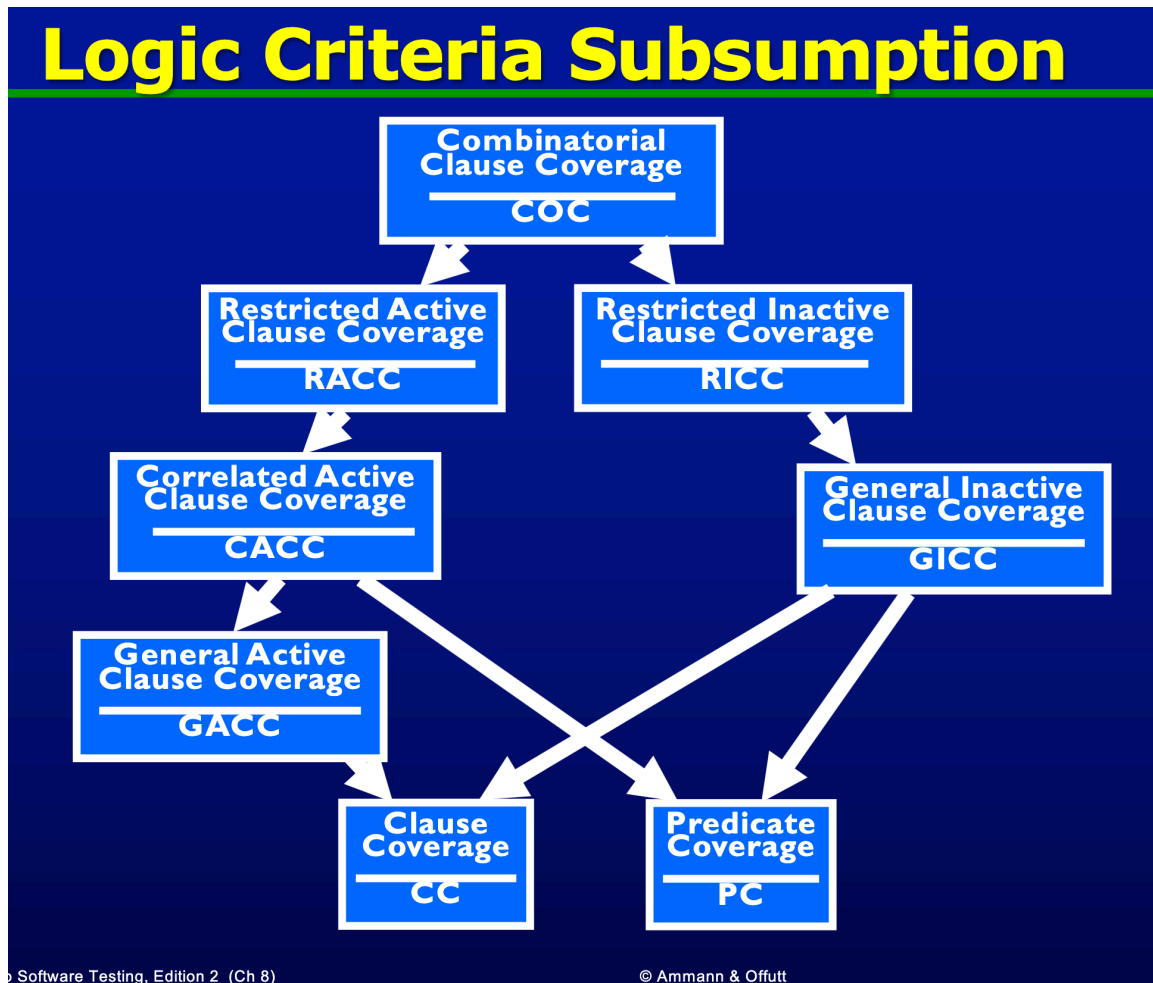




Chapter 8: Logic Coverage



0. Big Picture (What this chapter is REALLY about)

👉 Goal:

| Test **logical decisions** in software systematically

- Software decisions = **predicates**

- Each predicate = combination of **clauses**
- We cannot test all possibilities → choose **coverage criteria**

 Used in:

- Safety-critical systems (FAA requires it)
 - Games (EA), real-world systems
-

1. Core Concepts (FOUNDATION — MUST MASTER)

Predicate

- Boolean expression → true/false
 - Built from:
 - Variables
 - Comparisons
 - Boolean functions
 - Logical operators
-

Clause

- A **single atomic condition**

Example:

```
P = (a && (b || c))  
Clauses = a, b, c
```

 Real-world fact:

- 88.5% → 1 clause
- 9.5% → 2 clauses
- Very few ≥ 3 clauses

 Important insight:

Most real predicates are SIMPLE → but testing logic still tricky

◆ 2. Testing Model (IMPORTANT ABSTRACTION)

We convert software → logic model:

- P = set of predicates
- C_p = clauses inside predicate p
- TR = test requirements

👉 Testing = satisfying TR

◆ 3. Basic Coverage Criteria

✓ 3.1 Predicate Coverage (PC)

Definition

- Each predicate must be:
 - TRUE at least once
 - FALSE at least once

Example

```
P = (a && b)
```

```
Test 1: a=T, b=T → P=T
```

```
Test 2: a=F, b=F → P=F
```

! Weakness

- May ignore clauses

👉 Because:

- Short-circuit evaluation

- One clause dominates

📌 From slides:

| PC does not fully exercise all clauses

✅ 3.2 Clause Coverage (CC)

Definition

- Each clause must be:
 - TRUE
 - FALSE
-

! Weakness

- Does NOT guarantee predicate changes

👉 You can satisfy CC but:

- Predicate always TRUE

📌 Key exam trap:

| CC ≠ PC

🔥 PC vs CC (fixed)

Type	Requirement
PC	P = TRUE at least once and FALSE at least once
CC	each clause = TRUE at least once and FALSE at least once

💠 4. Strong Coverage

✅ 4.1 Combinatorial Coverage (CoC)

Definition

- Test ALL combinations of clauses

👉 If n clauses $\rightarrow 2^n$ tests

Example

```
P = (a & (b | c))  
→ 3 clauses → 8 tests
```

! Problems

- Exponential
 - Impractical if $n \geq 4$
-

💡 Key Insight

| CoC = strongest but too expensive

🔹 5. The REAL Problem

👉 Question:

| How to test each clause **independently affects the predicate?**

This leads to:

🔥 ACTIVE CLAUSE COVERAGE (MOST IMPORTANT PART)

🔹 6. Active Clause Concept (CRITICAL)

✅ Definition (MEMORIZE)

A clause **ci** **determines predicate p** if:

| Changing **ci** changes **p** (while others fixed)

Terms

- Major clause → clause being tested
 - Minor clauses → others
-

Intuition

 We want:

| "Does this clause actually matter?"

7. Active Clause Coverage (ACC)

ACC Definition

For each clause:

- Make it **determine predicate**
 - Test:
 - $ci = \text{true}$
 - $ci = \text{false}$
-

Ambiguity (VERY IMPORTANT)

Do minor clauses stay the same?

→ Leads to 3 variants:

Step 0 — Setup

Given predicate:

```
P = (a && (b || c))
```

Clauses = `{a, b, c}`

Step 1 — Pick MAJOR clause

Example:

```
Major = a
Minor = b, c
```

You will repeat this for **every clause**

Step 2 — Force clause to DETERMINE predicate

👉 Use this rule:

| Fix minors so that changing major flips P

7.1 GACC (General ACC)

- Minor clauses:
 - CAN change

! Problem:

- May NOT flip predicate

GACC (HOW TO DO)

Rule

- Clause determines predicate
 - Minor clauses can change
-

How to construct

1. Find ANY inputs where:

- $a = T \rightarrow \text{compute } P$
- $a = F \rightarrow \text{compute } P$

2. You do NOT need:

- same minors
 - predicate to flip
-

Key

| Easiest but weakest — don't overthink

7.2 RACC (Restricted ACC) (STRICTED)

- Minor clauses:
 - MUST stay same

 Problem:

- Often impossible

RACC (HOW TO DO)

Rule

- Clause determines predicate
 - Minor clauses MUST stay SAME
-

How to construct

1. Fix minors so clause determines P
2. Create:

```
(a=T, minors fixed)
(a=F, same minors)
```

1. Check:
 - P changes
-

If impossible

👉 Then RACC is **infeasible**

🔥 7.3 CACC (CORRELATED ACC) ★★ MOST IMPORTANT

Definition

- Clause determines predicate
 - Predicate **MUST** change value
-

💡 WHY IMPORTANT

- Equivalent to **MCDC (FAA standard)**
- Best balance of:
 - strength
 - feasibility

📌 From slides:

| CACC subsumes predicate coverage

🔥 EXAM RULE

👉 If asked:

| "Which coverage should we use?"

✅ Answer: **CACC**

CACC (MOST IMPORTANT — EXAM)

Rule

- Clause determines predicate
 - Predicate **MUST** change
-

✅ **Algorithm (USE THIS EVERY TIME)**

For each clause:

Step 1 — Fix minors so clause determines

Example:

```
P = a && (b || c)
Major = a

→ need (b || c) = TRUE
→ choose b = TRUE
```

Step 2 — Create 2 tests

```
Test 1: a = T, b = T → P = T
Test 2: a = F, b = T → P = F
```

Step 3 — Verify

- ✓ Only `a` changes
- ✓ Predicate changes
- 👉 DONE

✓ Correct (ACC)

```
b = FALSE

a = T → P = T
a = F → P = F
```

👉 `a` controls `P` ✓

🔥 Then what's the difference between GACC / RACC / CACC?

All 3 share:

👉 ✅ major clause determines P

Only difference:

Type	Minor clauses	Predicate must flip
GACC	can change	❌ no
CACC	can change	✅ YES 🔥
RACC	same	✅ YES (automatic)



Determine ≠ flip predicate

Determine = has an affect on the outcome of p

💠 8. How to Make a Clause Determine (CRITICAL SKILL)

✨ Method 1: Logical Reasoning

Example:

$P = A \text{ OR } B$

If $B = \text{false} \rightarrow A \text{ determines } P$

✨ Method 2: Definitional (EXAM LEVEL)

Steps:

1. Replace clause:

- $p(c=\text{true})$
- $p(c=\text{false})$

2. Compute:

```
pc = p(c=true) XOR p(c=false)
```

1. Solve → gives condition

📌 From slides:

| XOR gives exact condition for clause determination

✨ Method 3: Truth Table (MOST PRACTICAL)

- Find 2 rows where:
 - Only ci changes
 - Predicate changes

◆ 9. Inactive Clause Coverage (ICC)

✅ Idea (OPPOSITE OF ACC)

Clause should:

| NOT affect predicate

Requirements (4 tests per clause)

1. $c=T, p=T$
2. $c=F, p=T$
3. $c=T, p=F$
4. $c=F, p=F$

💡 Insight

- Tests independence of clause

◆ 10. Subsumption (VERY IMPORTANT)

👉 Which is stronger?

```
CoC
↑
RACC / RICC
↑
CACC
↑
GACC / GICC
↑
CC, PC
```

📌 Meaning:

- If you satisfy stronger → weaker automatically satisfied

💠 11. Source Code Challenges (Ch 8.3)

🔥 2 BIG PROBLEMS

1. Reachability

👉 Must reach code location first

Example (Thermostat):

```
if ((a || (b && c)) && d)
```

You must:

- Set inputs so execution reaches this line

2. Controllability

👉 Hard to control clause values because:

- Variables depend on program logic

Example:

```
dTemp = pSet.getSetting(...)
```

→ You must configure:

- period
- day
- settings

Key Insight

| In real code, clauses are NOT directly controllable

12. Real Example (Thermostat)

Predicate:

```
(a || (b && c)) && d
```

Where:

- a: temp condition
- b: override
- c: override temp condition
- d: timing condition

CACC Result

 Requires **6 tests**

(not 8 like CoC)

 From slide:

| "Six tests needed for CACC on Thermostat"

Insight

CACC:

- Much cheaper than CoC
- Still strong

13. Infeasible Test Cases

Definition

Some combinations:

| Cannot exist in reality

Example:

```
(a > b AND b > c AND c > a) → impossible
```

Rule

- Identify
- Ignore

 Important:

| Detecting infeasible tests is undecidable

14. Practical Guidelines (VERY USEFUL)

When to use what?

#Clauses	Best Coverage
1	PC
2-3	CoC
≥4	CACC

Real-world strategy

- Small predicates → brute force (CoC)
 - Large predicates → **CACC**
-

15. KEY EXAM INSIGHTS

Common traps

- ✗ CC guarantees PC → FALSE
 - ✗ PC guarantees CC → FALSE
 - ✗ GACC guarantees PC → FALSE
 - ✓ CACC guarantees PC → TRUE
-

Must memorize

- Definition of:
 - Predicate
 - Clause
 - Determination
 - Difference:
 - PC vs CC vs CoC vs CACC
 - How to:
 - Make clause determine predicate
-

FINAL SUMMARY

Core Idea

Test logic by:

| Controlling clause truth values



Best Practical Criterion



CACC (MCDC)



Reality

- CoC = strongest but impractical
 - PC/CC = too weak
 - CACC = best tradeoff
-